

Atelier 7 - API et connection pooling

Fiche étudiant • M2 • Jour 4 • Linux Bash • 2 h 15 (partie A : 1 h 15 ; partie B : 1 h)

Objectif et environnement

Durée : 2 h 15 (partie A : 1 h 15 ; partie B : 1 h). Travail individuel ou en binôme. Vous comparez deux paginations et deux stratégies de chargement sur l'API ShopFlow fournie. Vous vérifiez les données renvoyées, le nombre de requêtes SQL et les droits d'accès. Les stratégies sont déjà codées : votre travail consiste à les expérimenter et à expliquer leur fonctionnement.

Avant la séance : Linux, Bash, Node.js 22 ou supérieur, Docker avec Compose, dépendances installées et jeu initial ShopFlow chargé. Le dossier Jour4 se place dans optimisationBDD/02_Laboratoire. Le serveur se trouve dans optimisationBDD/01_server/api. Aucun PowerShell, jq ou curl n'est requis pour ces commandes.

Parcours de la séance

Partie A : pagination et chargement des relations, étapes 1 à 6. Partie B : mutualisation des connexions avec PgBouncer, étapes 7 à 12. Le dossier Atelier07_Nom réunit les preuves des deux parties. L'atelier 8 Redis reste séparé et inchangé. Prévoir 2 h 15 hors téléchargement et construction initiale des images.

1 Mise en route en 5 minutes

Dans .env, conserver LAB_CLIENT_ID=42 et vérifier LAB_TOKEN et CURSOR_SECRET. Dans un premier terminal placé dans 01_server/api, démarrer les services puis l'API. Garder ce terminal ouvert pour les traces.

```
docker compose up -d --wait
npm start
```

Dans un deuxième terminal Bash, se placer également dans 01_server/api puis charger l'aide avec source. Ne pas lancer le fichier avec sh ou dans un nouveau processus : ses fonctions doivent rester disponibles dans ce terminal.

```
source ../../02_Laboratoire/Jour4/Init_Jour4.sh
etat=$(sf demarrage /observations)
sf_champ "$etat" Body
```

Attendu : réponse 200. sf lit le jeton depuis .env et enregistre les appels réussis dans un CSV. La fonction renvoie Status, SqlCount, HttpMs, Traceld et Body. Redémarrer l'API après toute modification de .env.

Un appel sf affiche un objet JSON. La syntaxe est sf etiquette /chemin, ou sf etiquette /chemin PATCH corpsJSON. Pour conserver sa réponse dans une variable : resultat=\$(sf etiquette /chemin). Les messages d'erreur s'affichent séparément. En cas d'erreur, arrêter la séquence et vérifier le terminal API avant de poursuivre.

Contrôler le jeu du client 42 avant les manipulations.

```
docker compose exec -T postgres psql -U cours -d shopflow \
  -c 'SELECT count(*) FROM shopflow.commandes WHERE client_id=42;'
```

Attendu : 100 commandes, soit cinq pages de vingt sur un jeu stable. Chaque commande initiale possède trois lignes. Si le contrôle échoue, faire vérifier la préparation avant de poursuivre.

2 Comparaison de N plus 1 et du chargement groupé

Temps conseillé : 15 minutes. Demander les mêmes vingt commandes et leurs lignes avec les deux modes. L'ordre et les champs doivent rester identiques.

```

pagePath='/commandes?limit=20&pagination=curseur'
n1=$(sf n1_20 "${pagePath}&relations=n1")
groupe=$(sf groupe_20 "${pagePath}&relations=groupe")
sf_resume "$n1" "$groupe"
sf_liste "$n1"

```

Vérifier l'équivalence de toute la réponse, y compris les lignes. sf_champ extrait le corps JSON complet et sf_liste affiche le nombre de lignes de chaque commande.

```

if [[ "$(sf_champ "$n1" Body)" == "$(sf_champ "$groupe" Body)" ]]; then
    echo 'Réponses identiques'
else
    echo 'Réponses différentes'
fi
sf_liste "$groupe"

```

Contrôle	Résultat attendu
Statut HTTP	200
Nombre de commandes	20
Lignes par commande	3
Égalité des réponses	Réponses identiques
SQL avec relations=n1	21
SQL avec relations=groupe	2

Dans le terminal du serveur, retrouver les deux lignes JSON par Traceld. Relever sqlCount, sqlMs et durationMs. HttpMs côté client inclut le traitement HTTP du client Node.js et le transfert. sqlMs inclut l'attente éventuelle du pool. Ces durées ne mesurent pas exactement la même chose.

Dans commandes.mjs, repérer la boucle n1 et le chargement avec ANY du mode groupe. Expliquer le passage de 1 + N à 2 requêtes pour une page non vide. Une page vide ne déclenche qu'une requête.

Conserver les réponses et les traces. Une baisse du nombre de requêtes ne suffit pas à annoncer un gain de durée : reporter les valeurs réellement observées.

3 Comparaison de OFFSET et du curseur

Temps conseillé : 15 minutes. Avec le même jeu stable et le chargement groupé, comparer la deuxième page obtenue par OFFSET et par le curseur de la première page.

```

p1=$(sf curseur_p1 '/commandes?limit=20&relations=groupe')
cursor=$(sf_encoder "$(sf_champ "$p1" Body.nextCursor)")
nextPath="/commandes?limit=20&cursor=$cursor&relations=groupe"
p2Curseur=$(sf curseur_p2 "$nextPath")
offsetPath="/commandes?limit=20&pagination=offset&offset=20&relations=groupe"
p2Offset=$(sf offset_p2 "$offsetPath")
sf_resume "$p2Curseur" "$p2Offset"
if [[ "$(sf_champ "$p2Curseur" Body)" == "$(sf_champ "$p2Offset" Body)" ]]; then
    echo 'Réponses identiques'
else
    echo 'Réponses différentes'
fi

```

Attendu : Réponses identiques pour l'équivalence, 20 commandes et 2 requêtes SQL par appel. Le curseur provient de nextCursor de la réponse précédente. Le transmettre sans modification et encodé pour l'URL.

Écrire le contrat de l'endpoint : client autorisé provenant du contexte, ordre created_at DESC puis id DESC, limite par défaut 20 et maximum 100, réponse data / hasNextPage / nextCursor. Les identifiants et les montants sont des chaînes JSON. Les dates conservent six chiffres de microsecondes.

Prédicats SQL à expliquer

```
-- OFFSET, après SELECT ... FROM shopflow.commandes
WHERE client_id = $1
ORDER BY created_at DESC, id DESC
LIMIT 21 OFFSET 20;
```

```
-- Curseur, après SELECT ... FROM shopflow.commandes
WHERE client_id = $1 AND (created_at,id) < ($2,$3)
ORDER BY created_at DESC, id DESC
LIMIT 21;
```

La vingt-et-unième ligne sert à déterminer hasNextPage. Les relations sont ensuite chargées pour les vingt commandes effectivement renvoyées. Retrouver la traduction exacte dans commandes.mjs.

Les 100 commandes du client permettent surtout de vérifier le contrat. Ne pas annoncer un gain sur de grands décalages à partir d'un OFFSET qui dépasse le nombre de résultats.

4 Scénario des dates identiques

Temps conseillé : 20 minutes. Le jeu initial ne fournit pas d'égalité de dates pour le client 42. Le script sauvegarde les dates de 42, 1042 et 2042, puis leur attribue une même date située après les autres commandes de ce client. Les montants et les lignes restent identiques.

Depuis le deuxième terminal, dans 01_server/api, exécuter le script et vérifier son résultat.

```
docker compose exec -T postgres psql -U cours -d shopflow \
-v ON_ERROR_STOP=1 < ../../02_Laboratoire/Jour4/07_dates_identiques.sql
echo "Code de sortie SQL : $?"
```

La sortie SQL doit montrer 2042, 1042, 42 dans cet ordre, avec la même date. Réaliser deux pages de deux commandes : l'égalité doit traverser la frontière de page.

```
egalite1=$(sf_dates_p1 '/commandes?limit=2')
cursorDates=$(sf_encoder "${sf_champ "$egalite1" Body.nextCursor}")
egalite2=$(sf_dates_p2 "/commandes?limit=2&cursor=$cursorDates")
sf_liste "$egalite1"
sf_liste "$egalite2"
sf_champ "$egalite1" Body.hasNextPage
```

Page	Identifiants attendus
Première page, limite 2	2042 puis 1042
Deuxième page, limite 2	42 puis une autre commande du client
Contrôle	Les trois identifiants apparaissent une seule fois

Après 1042, une condition limitée à created_at < date_du curseur écarterait aussi 42, qui possède la même date. La comparaison du tuple (created_at,id) autorise 42, dont l'identifiant est plus petit dans l'ordre descendant. Expliquer ce défaut avec les réponses obtenues.

Conserver les deux réponses et leur nextCursor. Ne pas réinitialiser les données avant la restauration des dates à l'étape 5.

5 Contrôle du parcours et des droits

Temps conseillé : 10 minutes. Parcourir les cinq pages de vingt après le scénario des égalités. Aucune écriture ne doit intervenir pendant ce parcours.

```
idsParcours=(); cursorParcours=''; nbPages=0
while ;; do
  chemin='/commandes?limit=20&relations=groupe'
  if [[ -n "$cursorParcours" ]]; then
    chemin+="&cursor=$(sf_encoder "$cursorParcours")"
  fi
  page=$(sf "parcours_$nbPages" "$chemin") || break
  IFS=' ' read -r -a idsPage <<< "$(sf_ids "$page")"
  idsParcours+=("${idsPage[@]}")
  cursorParcours=$(sf_champ "$page" Body.nextCursor)
  nbPages=$((nbPages + 1))
  [[ -z "$cursorParcours" ]] && break
  if (( nbPages >= 10 )); then echo 'Parcours inattendu'; break; fi
done
uniques=$(printf '%s\n' "${idsParcours[@]}" | sort -u | wc -l)
printf 'Pages=%s Total=%s Uniques=%s\n' \
  "$nbPages" "${#idsParcours[@]}" "$uniques"
```

Attendu : Pages=5, Total=100, Uniques=100 sur ce jeu stable. Le curseur ne crée pas un instantané entre les appels HTTP.

Tester les deux refus suivants avec `sf_code`, qui accepte une réponse de refus et affiche son code HTTP. Les autres appels `sf` attendent une réponse réussie.

```
sf_code /commandes sans-jeton
sf_code '/commandes?client_id=43'
```

Attendu : 401 sans jeton, 400 lorsqu'un `client_id` est injecté dans l'URL. Ce contexte d'autorisation est pédagogique.

Restauration des dates

```
docker compose exec -T postgres psql -U cours -d shopflow \
  -v ON_ERROR_STOP=1 < ../../02_Laboratoire/Jour4/07_restaurer_dates.sql
echo "Code de sortie SQL : $?"
```

Attendu au premier passage : `dates_restorees=3`. La sauvegarde de ces trois dates est ensuite vidée. Conserver la sortie.

6 Dossier étudiant et bilan

Temps conseillé : 10 minutes. Remettre `Atelier07_Nom` : contrat de l'endpoint, fragments SQL expliqués, preuves et analyse. Référence du cours : Jour 4, diapositive 13.

Essai	SQL attendu	SQL observé	HTTP ms
N+1, page de 20	21	À relever	À relever
Goupé, page de 20	2	À relever	À relever
OFFSET, deuxième page	2	À relever	À relever
Curseur, deuxième page	2	À relever	À relever

Le CSV contient les mesures. Retrouver son emplacement avec `$J4_CSV`. Joindre les extraits du terminal API identifiés par Traceld, les deux pages de dates identiques, le bilan 100 / 100 / 5, les codes de refus et la sortie de restauration.

```
printf '%s\n' "$J4_CSV"  
cat "$J4_CSV"
```

Questions à traiter

1. Quel accès supplémentaire N+1 effectue-t-il pour chaque commande ? Montrer le passage concerné dans commandes.mjs.
2. Pourquoi charger les lignes après avoir déterminé les commandes de la page ? Quel problème une jointure avec LIMIT pourrait-elle créer ?
3. Pourquoi le curseur contient-il une date et un identifiant ? Utiliser le cas 2042 / 1042 / 42 pour expliquer le risque de perte.
4. Pourquoi un curseur signé ne remplace-t-il pas le contrôle d'accès ? Quelle est la source de client_id dans le kit ?
5. Que montrent les mesures sur votre machine ? Quelles conclusions demanderaient un jeu plus volumineux ou une campagne de charge ?

Critères de réussite

Les chargements renvoient les mêmes champs et relations. Le comptage confirme 21 puis 2 requêtes. Les pages de dates identiques contiennent les trois commandes sans doublon. Le parcours stable contient 100 identifiants uniques. Les droits sont conservés et les dates restaurées.

Les durées doivent provenir de vos appels. Une réduction théorique du nombre de requêtes ne constitue pas à elle seule une mesure de gain HTTP.

Atelier 7 - Complément PgBouncer

Fiche étudiant • Linux • Partie B • 60 minutes supplémentaires

7 Comprendre ce que le pool mutualise - 10 min

Objectif : mesurer le nombre de connexions PostgreSQL, identifier une attente de connexion et argumenter un dimensionnement. Cette partie complète le dossier Atelier07_Nom. Elle se rattache à B4C8 et à « Performance Backend : connection pooling (PgBouncer, HikariCP) » du programme du module, page 2. PgBouncer est l'outil choisi ; la compétence porte sur la gestion des données au service du back-end.

Chemin direct	Chemin mutualisé
Clients pgbench vers PostgreSQL	Clients pgbench vers PgBouncer vers PostgreSQL
Une session PostgreSQL par client connecté	Un ensemble limité de connexions PostgreSQL partagé entre les clients

Un pool applicatif, comme `pg.Pool` dans `Node.js`, réutilise les connexions à l'intérieur d'un processus. Plusieurs instances de l'API peuvent donc multiplier les connexions. PgBouncer est un service intermédiaire que ces instances peuvent partager. Les deux mécanismes peuvent coexister : régler l'un sans tenir compte de l'autre peut déplacer l'attente.

Mode transaction : une connexion serveur est affectée pendant la transaction et peut être réutilisée après sa fin. Ne pas confondre la connexion du client à PgBouncer et celle de PgBouncer à PostgreSQL. Une transaction longue conserve sa connexion et réduit la capacité disponible.

Un laboratoire séparé et contrôlé

Le complément fournit sa propre base « pooling », 1 000 produits et un rôle de lecture « atelier ». Aucun accès à la base ShopFlow, aucun changement de `.env`, de Redis ou de l'API. `pgbench` joue le rôle d'un client applicatif SQL : ces mesures ne sont pas des temps HTTP de ShopFlow.

Paramètre	Réglage fourni
PostgreSQL	<code>max_connections = 60</code>
PgBouncer	<code>pool_mode = transaction</code>
Connexions serveur	<code>default_pool_size = 5</code> , par couple base/utilisateur
Connexions clientes	<code>max_client_conn = 200</code>
Réserve de connexions	<code>reserve_pool_size = 0</code>
Charge courante / saturation	40 clients / 80 clients

Les scripts utilisent une seule base et un seul utilisateur de charge. La borne de 5 ne doit pas être généralisée à plusieurs couples base/utilisateur.

Préparer et contrôler

8 Démarrer le laboratoire - 10 min

Pré requis : Node.js 22 ou supérieur, Docker démarré et Docker Compose v2. Sous Windows, Docker Desktop doit utiliser les conteneurs Linux. Le premier démarrage télécharge les images et construit PgBouncer : prévoir Internet et effectuer cette préparation avant la séance.

Depuis le terminal de la partie A, encore placé dans 01_server/api :

```
cd ../../02_Laboratoire/Jour4/PgBouncer
node pooling.mjs up
```

Si le projet utilise un autre emplacement pour l'API, ouvrir directement le dossier 02_Laboratoire/Jour4/PgBouncer de votre copie du kit. Toutes les commandes suivantes se lancent dans ce dossier. Aucun npm install supplémentaire.

Le pilote Node appelle Docker avec des arguments structurés, identiques sur les deux systèmes. Les SQL et configurations sont copiés dans les images avec les droits nécessaires ; aucun dossier hôte n'est monté dans les conteneurs pour ces fichiers.

Contrôles avant toute mesure

```
node pooling.mjs check
```

Attendu : versions PostgreSQL, pgbench et PgBouncer affichées ; max_connections vaut 60 ; les chemins db:5432 et pgbouncer:6432 lisent chacun 1 000 produits. Dans SHOW CONFIG, vérifier pool_mode=transaction, default_pool_size=5 et max_client_conn=200. Le contrôle peut afficher de nombreuses lignes.

Si un contrôle échoue, arrêter ici. Exécuter la commande ci-dessous et conserver le message complet. Une erreur d'authentification ou de SQL ne doit pas être interprétée comme une saturation.

```
node pooling.mjs logs
```

Aucun port n'est publié sur l'hôte : 5432 et 6432 désignent les ports internes. Le laboratoire ne prend pas le port du PostgreSQL déjà utilisé par ShopFlow. Les identifiants en clair sont des identifiants de démonstration réservés à ce réseau Docker isolé.

Lire le scénario fourni

Ouvrir charge.sql. Chaque itération choisit un produit, ouvre une transaction, lit son prix, attend 50 ms avec pg_sleep, puis valide. Le client attend ensuite 100 ms hors transaction. Ces délais artificiels rendent visible la différence entre occupation d'un serveur et attente côté application. Ils ne simulent pas fidèlement une charge de production.

Question : pendant les 100 ms après COMMIT, quelle connexion reste ouverte dans chaque architecture, et laquelle peut être réutilisée ?

Comparer à charge identique

9 Mesurer les connexions et les délais - 15 min

```
node pooling.mjs compare --clients 40 --seconds 20 --repeats 3
```

La campagne exécute trois essais par chemin, en alternant leur ordre. Avant chacun : échauffement de 5 secondes avec 4 clients, hors des mesures. PgBouncer est arrêté pour le chemin direct afin que ses connexions inactives ne faussent pas le comptage. Il est relancé avant les essais via le pool et à la fin de la campagne.

Le pilote conserve les sorties pgbench et échantillonne pg_stat_activity environ une fois par seconde, avec un surcoût Docker. Les observations ne donnent donc pas un maximum instantané exact. Le filtre compte seulement le rôle atelier dans la base pooling ; la connexion de supervision PostgreSQL n'est pas comptée.

Les fichiers sont enregistrés dans PgBouncer/resultats, dans un nouveau sous-dossier horodaté à chaque campagne.

Fichier	À exploiter
synthese.csv	TPS, latence moyenne, code de sortie, transactions, pic échantillonné
*_pgbench.txt	Sortie brute, erreurs éventuelles et progression
*_connexions.csv	Total, active, idle, idle_in_transaction au fil du temps
*_observations.txt	SHOW POOLS pendant la charge ; SHOW STATS final
configuration.txt	Versions, images et réglages de la campagne

Résultats structurels à vérifier

Avec 40 clients persistants, le chemin direct garde normalement environ 40 sessions pour atelier. Le chemin mutualisé limite les connexions serveur de ce couple à 5 ; le contrôle de santé utilise ce même couple. Dans SHOW POOLS, retenir la ligne pooling / atelier, pas la console administrative pgbouncer.

sv_active indique les serveurs liés à un client ; sv_idle ceux immédiatement disponibles. cl_waiting compte les clients attendant une connexion serveur. cl_active ne désigne pas uniquement des clients exécutant du SQL : il inclut aussi des clients inactifs sans requête en attente. maxwait renseigne l'attente la plus longue parmi les clients en attente.

Moins de connexions ne signifie pas automatiquement moins de latence. Avec seulement 5 serveurs pour 40 clients et 50 ms d'occupation par transaction, des clients attendent. PgBouncer peut être plus lent dans ce scénario tout en limitant le nombre de sessions PostgreSQL.

Coût des ouvertures répétées

```
node pooling.mjs compare --clients 10 --seconds 20 --repeats 3 --connect
```

Cette campagne ajoute l'option -C de pgbench : chaque itération ouvre une nouvelle connexion cliente. Avec PgBouncer, les connexions serveur sont réutilisées, mais l'authentification cliente et le passage par le proxy restent des coûts. Comparer direct et pool au sein de cette campagne ; ne pas attribuer au seul -C les différences avec la campagne à 40 clients.

Observer la saturation

10 Dépasser la limite des sessions - 10 min

```
node pooling.mjs saturation --seconds 20
```

Le pilote lance 80 clients persistants dans chaque chemin. PostgreSQL est limité à 60 connexions totales, dont certaines sont réservées. Le rôle atelier n'est pas superutilisateur. En direct, une erreur de limite de connexions est donc attendue ; la charge peut s'interrompre dès l'ouverture des sessions.

Avec PgBouncer, 80 connexions clientes restent sous `max_client_conn=200` ; 5 connexions serveur sont partagées. On attend une exécution sans refus pour limite PostgreSQL, avec de l'attente côté PgBouncer. Si un autre incident survient, le diagnostiquer avec les traces.

Chemin	Observation attendue	Preuve
Direct, 80 clients	Refus de connexions et possible arrêt de pgbench	Message FATAL / limite dans *_pgbench.txt et code de sortie
PgBouncer, 80 clients	Clients servis successivement, au plus 5 serveurs pour atelier	Connexions échantillonnées et SHOW POOLS pendant la charge

Le pilote tolère un code non nul pour cet essai direct afin d'exécuter ensuite le chemin mutualisé. Cela ne valide pas la cause : lire le message. Une campagne échouée ne fournit pas un débit comparable à celui d'une campagne complète. Si la latence ou les TPS manquent dans le CSV, noter « non mesurable », pas zéro.

Lire les observations pendant la charge

Les fichiers *_observations.txt sont la preuve principale. Pour une observation manuelle, ouvrir un deuxième terminal dans le même dossier pendant la phase PgBouncer :

```
node pooling.mjs observe
```

Un échec de cette commande pendant la phase directe est normal : PgBouncer y est arrêté volontairement. Après la charge, `cl_waiting` peut redevenir nul ; cela ne démontre pas l'absence d'attente pendant le test. SHOW STATS est cumulatif depuis le démarrage du processus : il peut inclure l'échauffement ou un essai précédent. Utiliser les sorties pgbench pour les mesures de chaque essai.

Limites à expliquer

Le pool ne rend pas la requête SQL plus rapide et ne corrige pas le N+1. Il encadre l'utilisation des connexions. Une file d'attente peut remplacer un refus, mais ne crée pas de capacité de traitement. Un pool trop petit augmente l'attente ; un pool trop grand peut surcharger le serveur.

En mode transaction, ne pas supposer qu'un état de session suit le client entre deux transactions. Les paramètres de session, tables temporaires ou préparations nommées exigent une vérification de compatibilité selon la version et le pilote. Le test fourni utilise des transactions explicites et le protocole simple pour éviter cette dépendance.

Restituer et terminer

11 Dossier Atelier07_Nom - 10 min

Compléter le dossier de la partie A avec les CSV, les traces d'au moins une attente, la preuve du refus direct à 80 clients et la configuration. Pour chaque campagne réussie, présenter les trois valeurs puis leur médiane. Une campagne de 20 secondes est une démonstration pédagogique courte, pas une preuve de capacité de production.

Campagne	Connexions serveur direct / pool	TPS direct / pool	Latence direct / pool
40 clients persistants	À mesurer	À mesurer	À mesurer
10 clients, reconnexion	À mesurer	À mesurer	À mesurer
80 clients, saturation	À observer	Non comparable si essai direct interrompu	Même réserve

Préciser si la latence pgbench inclut le scénario et ses attentes : ce n'est ni le temps du seul SELECT, ni un temps HTTP. Ne pas utiliser `duree_pilote_s` comme latence : cette colonne est une durée de pilotage et d'observation. Une case `transactions_echouees` vide signifie que pgbench n'a pas fourni cette métrique, pas qu'aucune erreur n'est survenue.

Questions à traiter :

1. Quelle différence entre `pg.Pool` dans chaque processus Node.js et PgBouncer partagé ? Que pourrait produire le déploiement de dix instances de l'API ?
2. Comment 80 connexions clientes peuvent-elles être servies par 5 connexions PostgreSQL ? Montrer la frontière de transaction dans `charge.sql`.
3. Quel résultat prouve une diminution du nombre de connexions ? Quel résultat prouve une attente ? Ces résultats prouvent-ils un gain de temps de réponse ?
4. Pourquoi faut-il encore corriger le N+1 et les index avec PgBouncer ? Quel réglage tester ensuite, et quelle métrique surveiller pour éviter de déplacer le problème ?

Critères : contrôles initiaux réussis, mesures réelles conservées, distinction client/serveur, refus direct correctement identifié, attente documentée, limites du test explicitées. Aucune accélération chiffrée n'est imposée.

12 Arrêt et retour à l'atelier 8

```
node pooling.mjs down
cd ../../../../01_server/api
```

down arrête et retire les conteneurs du seul projet `shopflow-j4-pooling` ; le volume reste disponible. Reprendre ensuite la fiche de l'atelier 8, inchangée. Ne pas modifier son TTL ou son fonctionnement pour ce complément.

Si le parcours de votre projet diffère, revenir au dossier réel contenant `server.mjs` et `.env`. Après Ctrl+C pendant une charge, lancer `down` avant `up` afin d'éliminer les processus interrompus. Pour repartir de zéro sur ce laboratoire uniquement : `node pooling.mjs reset --confirm`, puis `node pooling.mjs up`. Cette remise à zéro supprime ses données de démonstration.

Sources techniques : pgbouncer.org/config.html ; pgbouncer.org/usage.html ; postgresql.org/docs/17/pgbench.html. Programme pédagogique : « DEV - RNCP 40166 - M2 - OPTIMISATION BDD - M2 », page 2. Consultation : 7 octobre 2026.